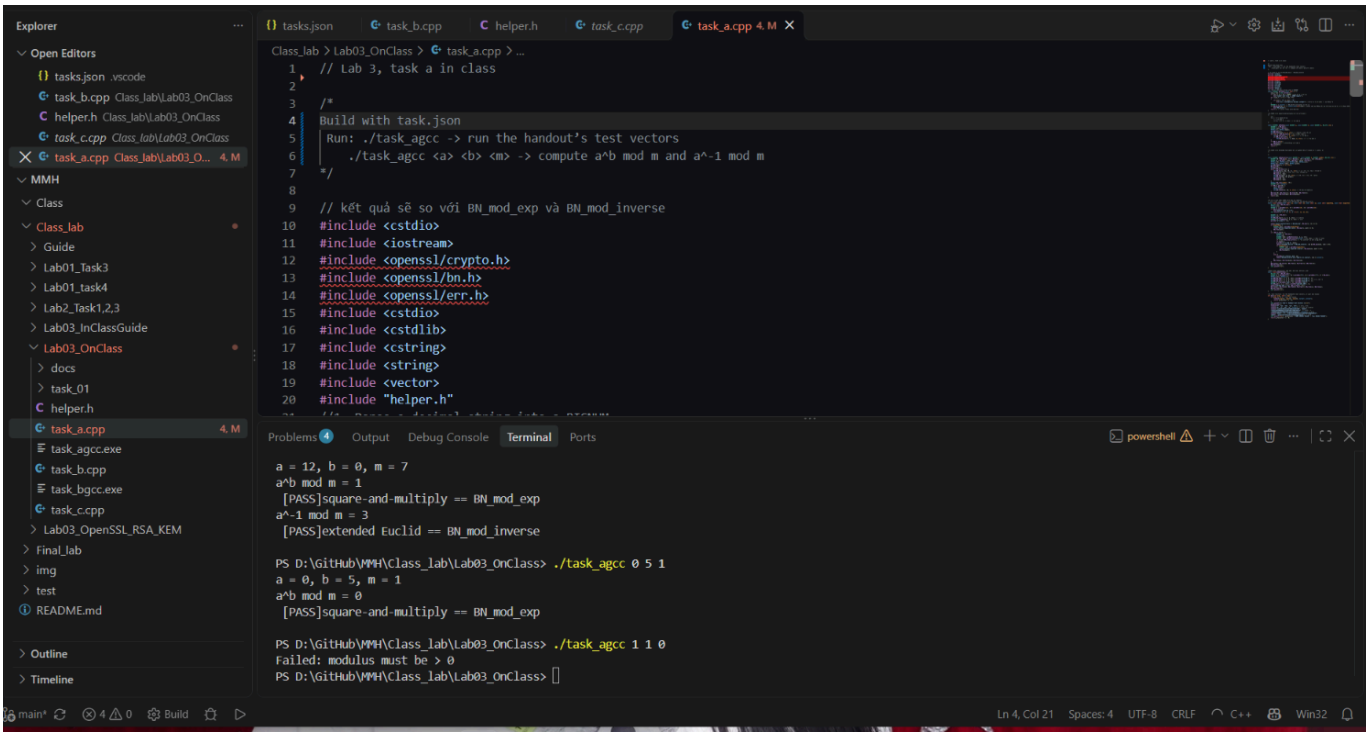
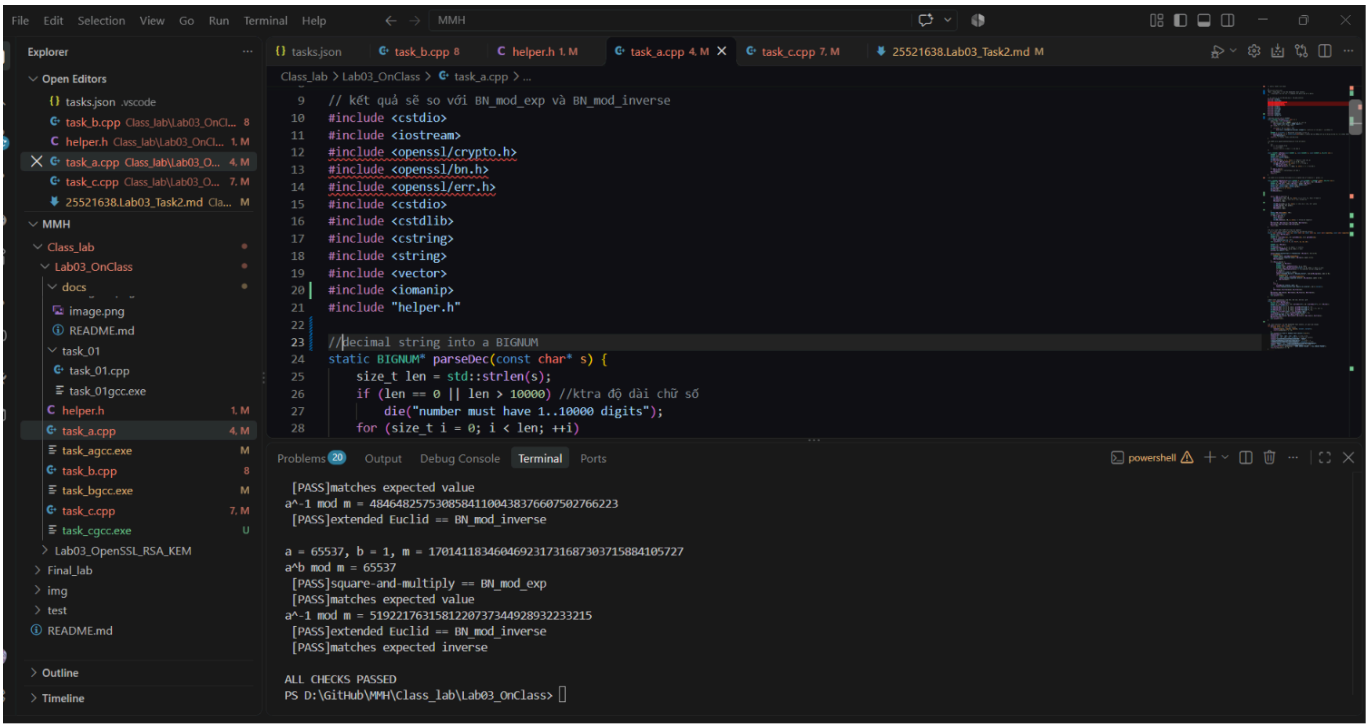
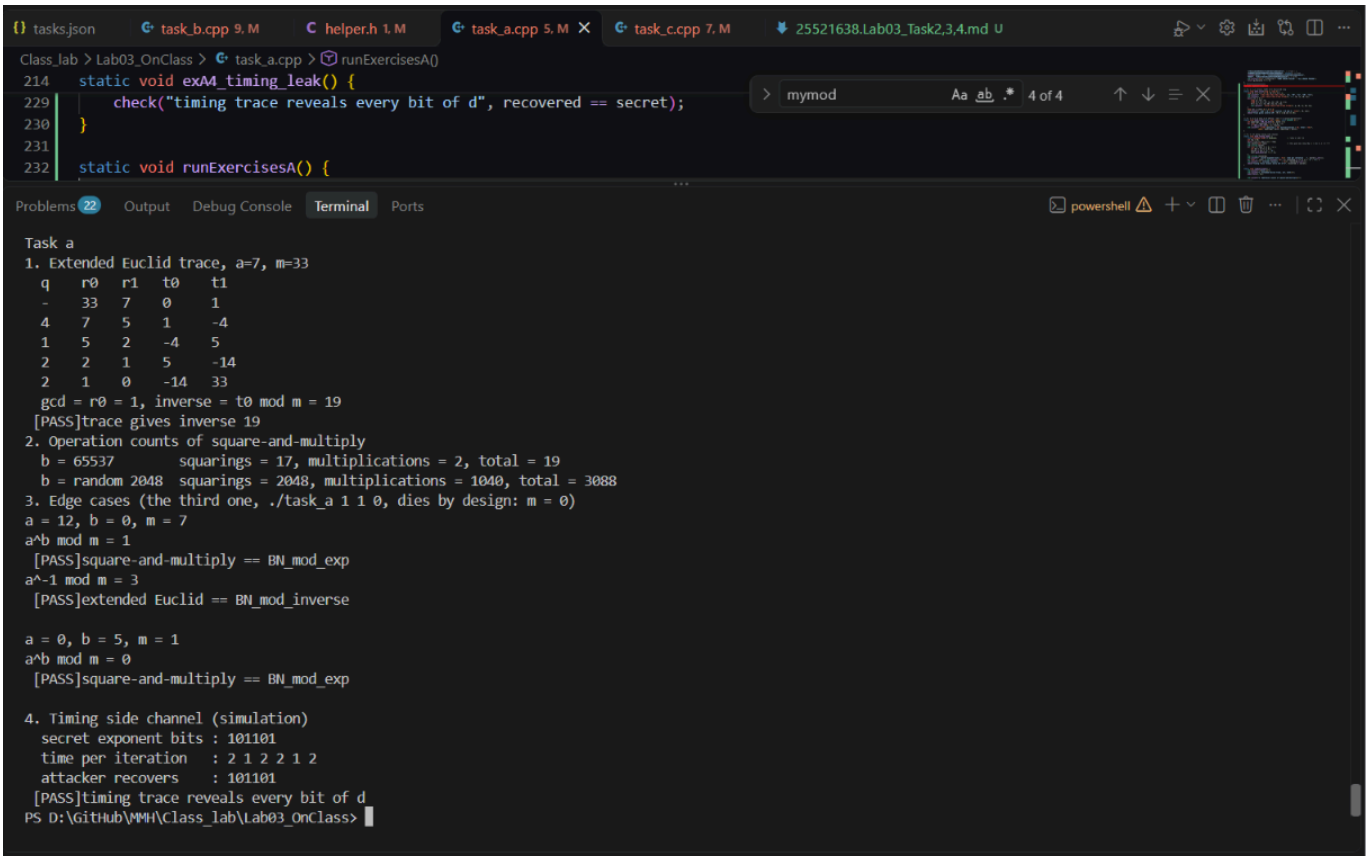
Task a


// Lab 3, task a in class
/*
Build with task.json
Run: ./task_agcc -> run the handout's test vectors
./task_agcc <a> <b> <m> -> compute a^b mod m and a^-1 mod m
*/
// kết quả sẽ so với BN_mod_exp và BN_mod_inverse
#include <cstdio>
#include <iostream>
#include <openssl/crypto.h>
#include <openssl/bn.h>
#include <openssl/err.h>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <string>
#include <vector>
#include "helper.h"
a = 12, b = 0, m = 7
a^b mod m = 1
[PASS]square-and-multiply == BN_mod_exp
a^-1 mod m = 3
[PASS]extended Euclid == BN_mod_inverse
PS D:\GitHub\MMH\Class_lab\Lab03_OnClass> ./task_agcc 0 5 1
a = 0, b = 5, m = 1
a^b mod m = 0
[PASS]square-and-multiply == BN_mod_exp
PS D:\GitHub\MMH\Class_lab\Lab03_OnClass> ./task_agcc 1 1 0
Failed: modulus must be > 0
PS D:\GitHub\MMH\Class_lab\Lab03_OnClass>


// kết quả sẽ so với BN_mod_exp và BN_mod_inverse
#include <cstdio>
#include <iostream>
#include <openssl/crypto.h>
#include <openssl/bn.h>
#include <openssl/err.h>
#include <cstdio>
#include <cstdlib>
#include <cstring>
#include <string>
#include <vector>
#include <iomanip>
#include "helper.h"
//decimal string into a BIGNUM
static BIGNUM* parseDec(const char* s) {
size_t len = std::strlen(s);
if (len == 0 || len > 10000) //ktra độ dài chữ số
die("number must have 1..10000 digits");
for (size_t i = 0; i < len; ++i)
[PASS]matches expected value
a^-1 mod m = 484648257530858411004383766075027662 23
[PASS]extended Euclid == BN_mod_inverse
a = 65537, b = 1, m = 170141183460469231731687303715884105727
a^b mod m = 65537
[PASS]square-and-multiply == BN_mod_exp
[PASS]matches expected value
a^-1 mod m = 519221763158122073734492893223321 5
[PASS]extended Euclid == BN_mod_inverse
[PASS]matches expected inverse
ALL CHECKS PASSED
PS D:\GitHub\MMH\Class_lab\Lab03_OnClass>



Answer

Bài 1: vết Euclid kết thúc với r0 = 1, t0 = -14 -> nghịch đảo là -14 mod 33 = 19.

Bài 2: với 65537 = 2^{16}+1 chỉ có 17 phép bình phương và 2 phép nhân, vì chỉ có 2 bit 1(10000000000000001).

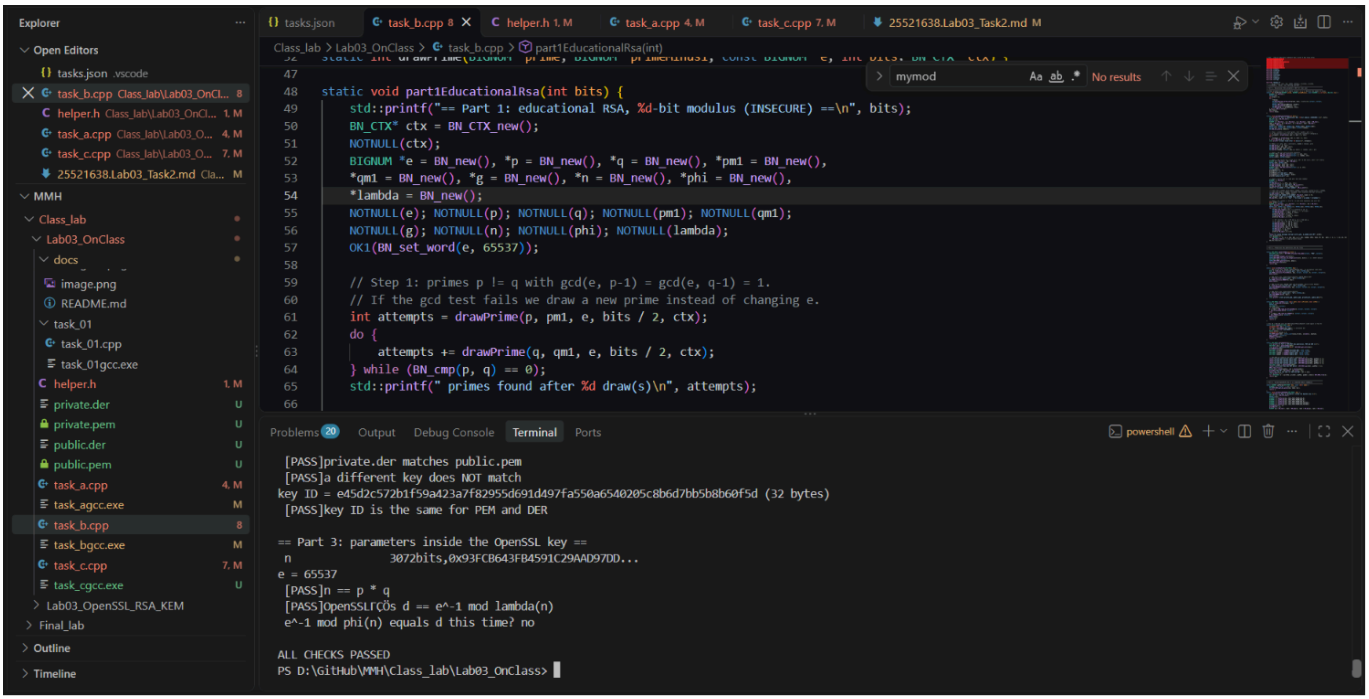
Số mũ 2048 bit ngẫu nhiên cần khoảng 2048 bình phương và khoảng 1024 phép nhân -> lý do mã hóa/xác minh với e = 65537 nhanh.

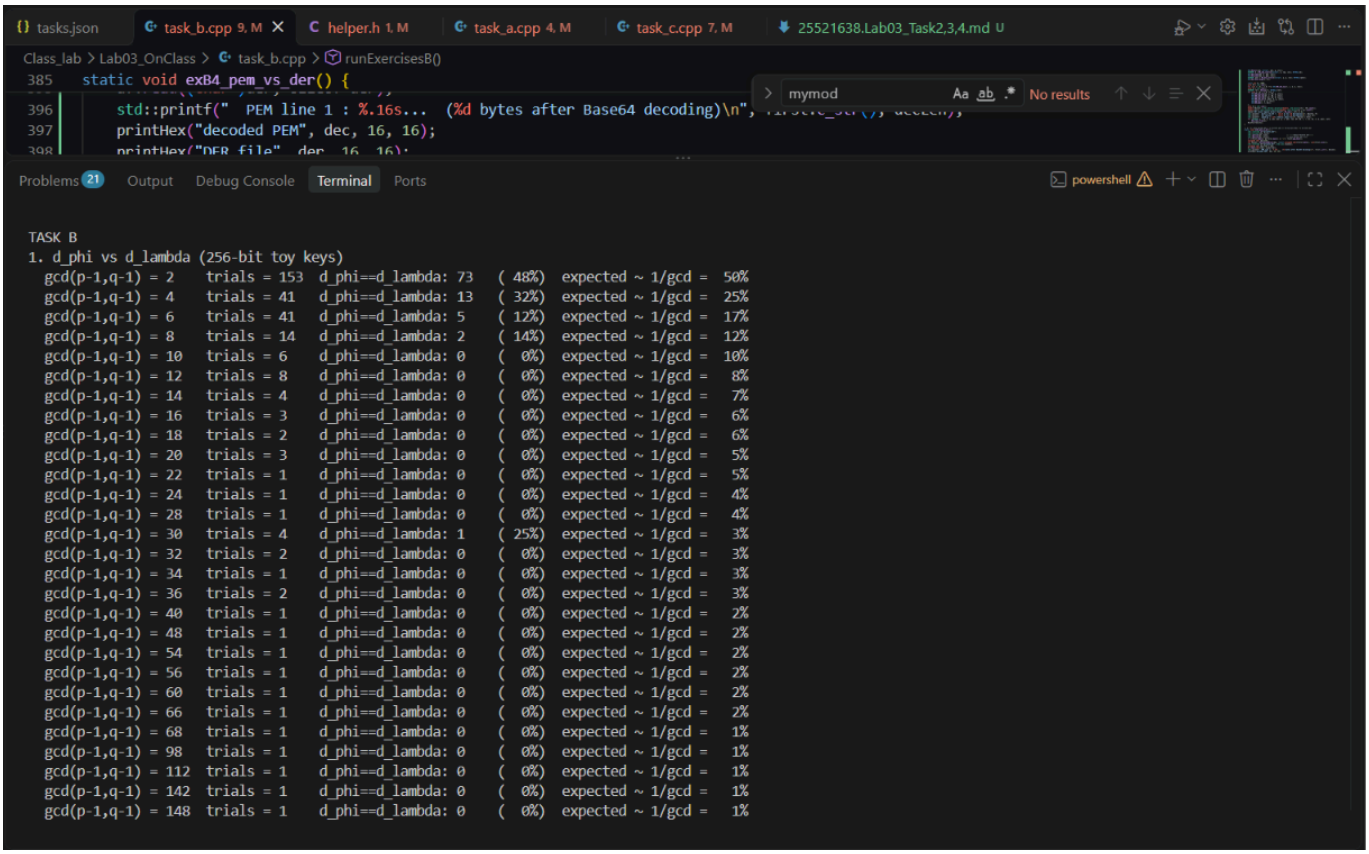
Bài 3:

- 12 0 7: xác định, kết quả 12^0 = 1 và nghịch đảo của 12 mod 7 là 3.
- 0 5 1: xác định, mọi thứ đều bằng 0 mod 1. Chương trình bỏ qua phần nghịch đảo.
- 1 1 0: m = 0 thì không xác định, chương trình dừng với FAILED: modulus must be > 0.

Bài 4: bit 1 làm vòng lặp lâu hơn (bình phương + nhân), bit 0 nhanh hơn. Kẻ tấn công đo thời gian từng vòng sẽ đọc ra từng bit của d. Cách phòng chống là dùng thuật toán thời gian hằng số khiến kẻ gian không phân biệt được. Vd khóa thật của OpenSSL.

Task b


Explorer
Open Editors
tasks.json .vscode
task_b.cpp Class_lab\Lab03_OnCl... 8
helper.h Class_lab\Lab03_OnCl... 1, M
task_a.cpp Class_lab\Lab03_O... 4, M
task_c.cpp Class_lab\Lab03_O... 7, M
25521638.Lab03_Task2.md Cla... M
MMH
Class_lab
Lab03_OnClass
docs
image.png
README.md
task_01
task_01.cpp
task_01gcc.exe
helper.h 1, M
private.der U
private.pem U
public.der U
public.pem U
task_a.cpp 4, M
task_agcc.exe M
task_b.cpp 8
task_bgcc.exe M
task_c.cpp 7, M
task_cgcc.exe U
Lab03_OpenSSL_RSA_KEM
Final_lab
Outline
Timeline
tasks.json
task_b.cpp 8
helper.h 1, M
task_a.cpp 4, M
task_c.cpp 7, M
25521638.Lab03_Task2.md M
Class_lab > Lab03_OnClass > task_b.cpp > part1EducationalRsa(int)
mymod
No results
47
48 static void part1EducationalRsa(int bits) {
49 std::printf("== Part 1: educational RSA, %d-bit modulus (INSECURE) ==\n", bits);
50 BN_CTX* ctx = BN_CTX_new();
51 NOTNULL(ctx);
52 BIGNUM *e = BN_new(), *p = BN_new(), *q = BN_new(), *pm1 = BN_new(),
53 *qm1 = BN_new(), *g = BN_new(), *n = BN_new(), *phi = BN_new(),
54 *lambda = BN_new();
55 NOTNULL(e); NOTNULL(p); NOTNULL(q); NOTNULL(pm1); NOTNULL(qm1);
56 NOTNULL(g); NOTNULL(n); NOTNULL(phi); NOTNULL(lambda);
57 OK1(BN_set_word(e, 65537));
58
59 // Step 1: primes p != q with gcd(e, p-1) = gcd(e, q-1) = 1.
60 // If the gcd test fails we draw a new prime instead of changing e.
61 int attempts = drawPrime(p, pm1, e, bits / 2, ctx);
62 do {
63 attempts += drawPrime(q, qm1, e, bits / 2, ctx);
64 } while (BN_cmp(p, q) == 0);
65 std::printf(" primes found after %d draw(s)\n", attempts);
66
Problems 20
Output
Debug Console
Terminal
Ports
powershell
[PASS]private.der matches public.pem
[PASS]a different key does NOT match
key ID = e45d2c572b1f59a423a7f82955d691d497fa550a6540205c8b6d7bb5b8b60f5d (32 bytes)
[PASS]key ID is the same for PEM and DER
== Part 3: parameters inside the OpenSSL key ==
n 3072bits,0x93FCB643FB4591C29AAD97DD...
e = 65537
[PASS]n == p * q
[PASS]OpenSSLΓÇÖs d == e^-1 mod lambda(n)
e^-1 mod phi(n) equals d this time? no
ALL CHECKS PASSED
PS D:\GitHub\MMH\Class_lab\Lab03_OnClass>


tasks.json
task_b.cpp 9, M
helper.h 1, M
task_a.cpp 4, M
task_c.cpp 7, M
25521638.Lab03_Task2,3,4.md U
Class_lab > Lab03_OnClass > task_b.cpp > runExercisesB()
mymod
No results
385 static void exB4_pem_vs_der() {
396 std::printf(" PEM line 1 : %.16s... (%d bytes after Base64 decoding)\n",
397 printHex("decoded PEM", dec, 16, 16);
Problems 21
Output
Debug Console
Terminal
Ports
powershell
TASK B
1. d_phi vs d_lambda (256-bit toy keys)
gcd(p-1,q-1) = 2 trials = 153 d_phi==d_lambda: 73 (48%) expected ~ 1/gcd = 50%
gcd(p-1,q-1) = 4 trials = 41 d_phi==d_lambda: 13 (32%) expected ~ 1/gcd = 25%
gcd(p-1,q-1) = 6 trials = 41 d_phi==d_lambda: 5 (12%) expected ~ 1/gcd = 17%
gcd(p-1,q-1) = 8 trials = 14 d_phi==d_lambda: 2 (14%) expected ~ 1/gcd = 12%
gcd(p-1,q-1) = 10 trials = 6 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 10%
gcd(p-1,q-1) = 12 trials = 8 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 8%
gcd(p-1,q-1) = 14 trials = 4 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 7%
gcd(p-1,q-1) = 16 trials = 3 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 6%
gcd(p-1,q-1) = 18 trials = 2 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 6%
gcd(p-1,q-1) = 20 trials = 3 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 5%
gcd(p-1,q-1) = 22 trials = 1 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 5%
gcd(p-1,q-1) = 24 trials = 1 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 4%
gcd(p-1,q-1) = 28 trials = 1 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 4%
gcd(p-1,q-1) = 30 trials = 4 d_phi==d_lambda: 1 (25%) expected ~ 1/gcd = 3%
gcd(p-1,q-1) = 32 trials = 2 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 3%
gcd(p-1,q-1) = 34 trials = 1 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 3%
gcd(p-1,q-1) = 36 trials = 2 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 3%
gcd(p-1,q-1) = 40 trials = 1 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 2%
gcd(p-1,q-1) = 48 trials = 1 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 2%
gcd(p-1,q-1) = 54 trials = 1 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 2%
gcd(p-1,q-1) = 56 trials = 1 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 2%
gcd(p-1,q-1) = 60 trials = 1 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 2%
gcd(p-1,q-1) = 66 trials = 1 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 2%
gcd(p-1,q-1) = 68 trials = 1 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 1%
gcd(p-1,q-1) = 98 trials = 1 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 1%
gcd(p-1,q-1) = 112 trials = 1 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 1%
gcd(p-1,q-1) = 142 trials = 1 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 1%
gcd(p-1,q-1) = 148 trials = 1 d_phi==d_lambda: 0 (0%) expected ~ 1/gcd = 1%

```
  gcd(p-1,q-1) = 112  trials = 1    d_phi==d_lambda: 0    (  0%)  expected ~ 1/gcd =   1%
  gcd(p-1,q-1) = 142  trials = 1    d_phi==d_lambda: 0    (  0%)  expected ~ 1/gcd =   1%
  gcd(p-1,q-1) = 148  trials = 1    d_phi==d_lambda: 0    (  0%)  expected ~ 1/gcd =   1%
  gcd(p-1,q-1) = 156  trials = 1    d_phi==d_lambda: 1    (100%)  expected ~ 1/gcd =   1%
  gcd(p-1,q-1) = 948  trials = 1    d_phi==d_lambda: 0    (  0%)  expected ~ 1/gcd =   0%
  overall: 95 / 300 trials equal
2. Prime draws: e = 3 vs e = 65537
  e = 3      -> 1.89 draws per prime (over 300 primes)
  e = 65537  -> 1.00 draws per prime (over 300 primes)
  e = 3 fails whenever p = 1 (mod 3), i.e. for about half of all primes.
3. Direct decryption vs CRT (1024-bit toy key)
 [PASS]CRT result equals direct result
  direct c^d mod n :    454.4 ms for 1000 decryptions
  CRT (Garner)     :    150.3 ms for 1000 decryptions
  speed-up = 3.02x (theory: about 4x)
4. PEM Base64 vs DER bytes
  PEM line 1 : MIIG/AIBADANBgkq...  (48 bytes after Base64 decoding)
decoded PEM = 308206fc020100300d06092a864886f7 (16 bytes)
DER file = 308206fc020100300d06092a864886f7 (16 bytes)
  'MII' = 001100 001000 001000 -> 0x30 0x82 = ASN.1 SEQUENCE with a 2-byte length
 [PASS]first Base64 line == first bytes of private.der
```

Answer

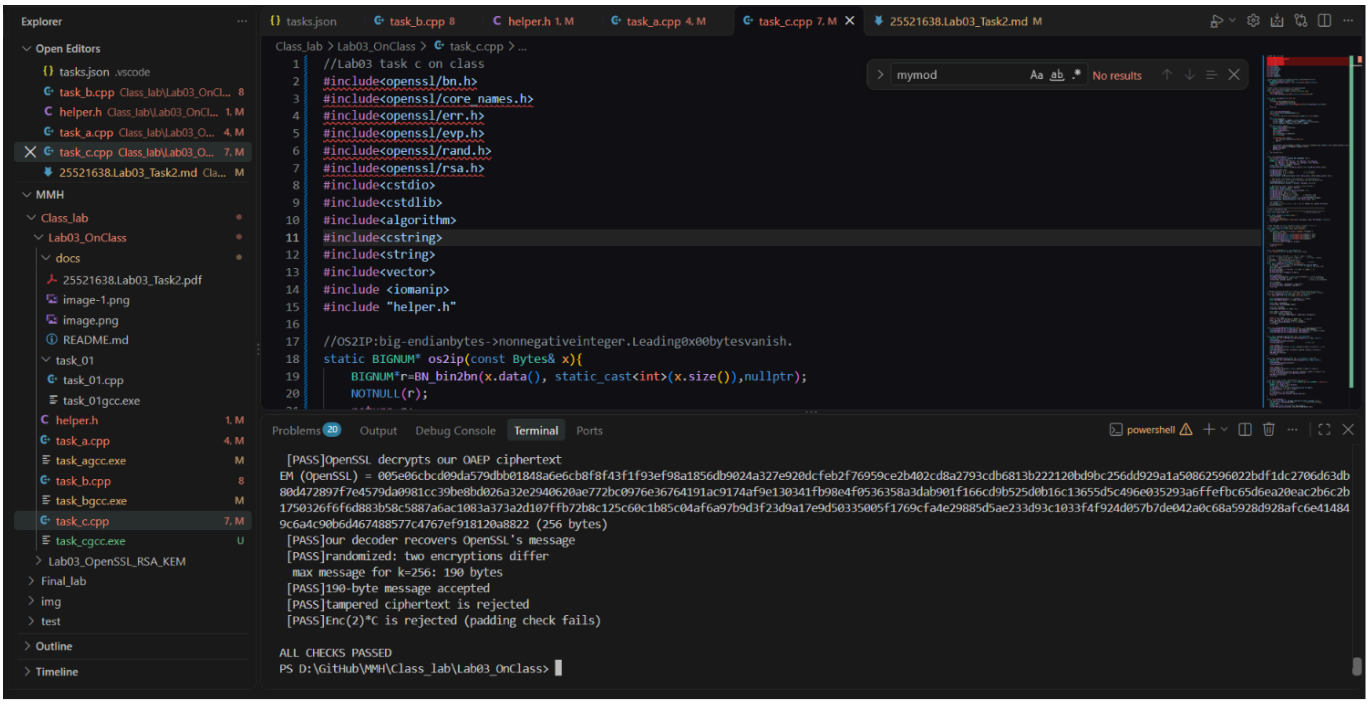
Bài 1: p−1 và q−1 đều chẵn nên gcd luôn >= 2. d_φ = d_λ + k·λ với k nằm trong $[0, gcd)$, nên xác suất hai số bằng nhau xấp xỉ 1/gcd. Với gcd = 2 là khoảng 50%.

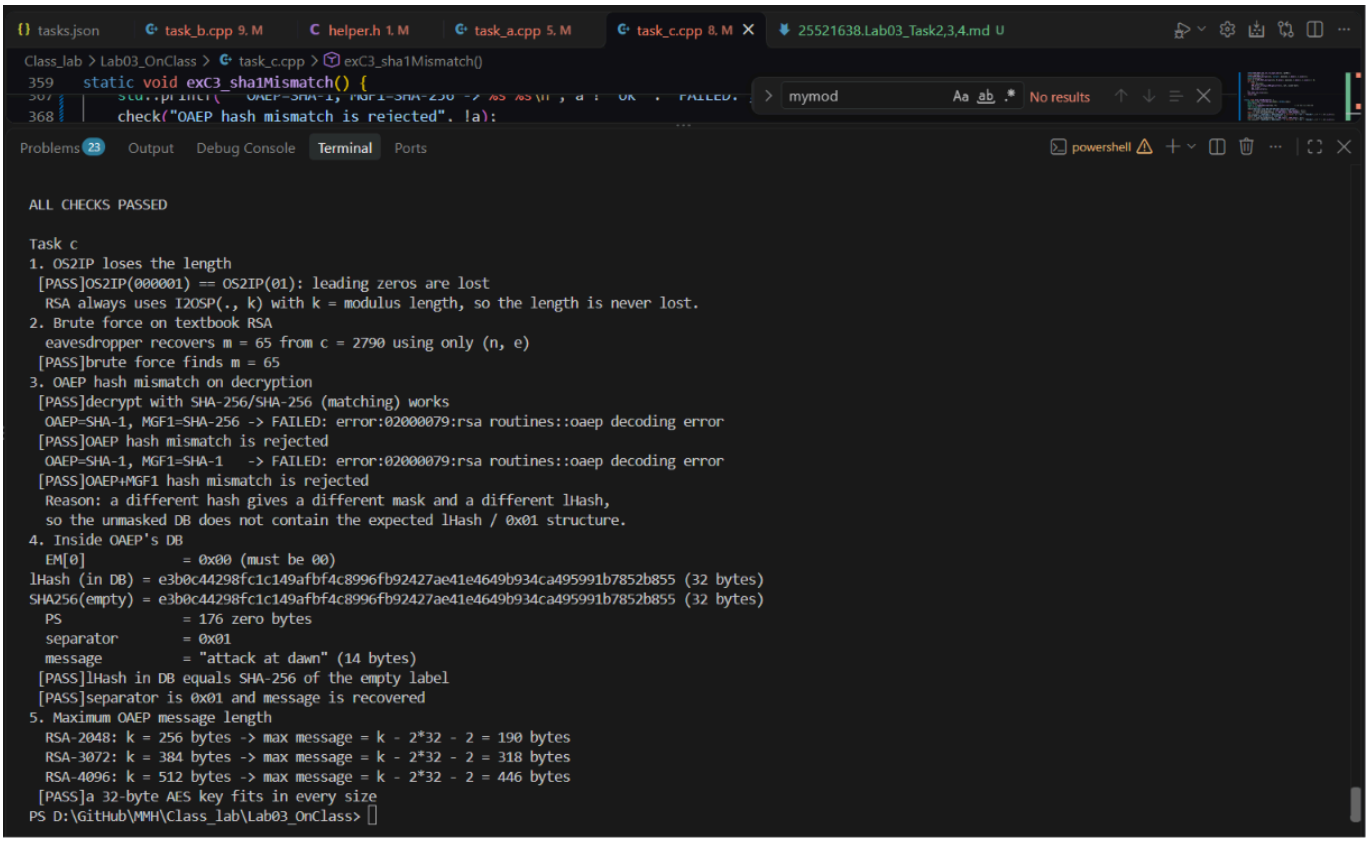
Bài 2: e = 65537 cần khoảng 1 lần rút. e = 3 cần khoảng 2 lần, vì p ≡ 1 (mod 3) xảy ra với khoảng một nửa số nguyên tố.

Bài 3: CRT thường nhanh hơn khoảng 3–4 lần, gần với dự đoán lý thuyết là 4 lần (hai phép mũ trên số mũ và modulus nửa kích thước).

Bài 4: MII giải mã Base64 thành 30 82, là byte SEQUENCE của ASN.1, khớp với đầu file DER.

Task c





Answer

Bài 1: 000001 và 01 đều cho OS2IP = 1, nên OS2IP làm mất thông tin độ dài. RSA luôn dùng I2OSP với độ dài cố định k nên không gặp vấn đề này.

Bài 2: vòng lặp tìm ra m = 65. Mã hóa tất định nên ai cũng thử đoán rồi so sánh với bản mã được.

Bài 3: giải mã bằng SHA-1 trong khi mã hóa bằng SHA-256 báo oaep decoding error. Hash khác nên lHash và mặt nạ MGF1 khác, DB sau khi gỡ mặt nạ chỉ là dữ liệu rác.

Bài 4: DB gồm lHash (e3b0c442...), PS gồm 176 byte 0, byte phân cách 0x01, rồi đến “attack at dawn”.

Bài 5: tối đa 190 byte (2048 bit), 318 byte (3072 bit) và 446 byte (4096 bit). Khóa AES 32 byte luôn nhỏ hơn các giới hạn này.